Universidad del Valle de Guatemala

CC2008 - Introducción a la Programación Orientada a Objetos

Nery Javier de la Cruz Huinil - 261233

David Maximiliano Pablo Cuy - 261341

Análisis:

1. Cuadro de propiedades

Propiedades					
#	tipoDato	nombre	visibilidad	motivo	Clase
1	String	codigo	private	Guarda el código que identifica a cada envío. No puede estar vacío ni repetirse, y es el dato con el que se busca el envío para asignarlo, retirarlo o registrar su entrega.	Envio
2	String	remitente	private	Guarda el nombre de la persona que manda el envío. Se muestra al consultarlo y es a quien se le devuelve si se agotan los intentos.	Envio
3	String	destinatario	private	Guarda el nombre de la persona que va a recibir el envío, para mostrarlo en las consultas.	Envio
4	double	pesoReal	private	Guarda el peso real en kilogramos, que puede tener decimales. Tiene que ser mayor que 0 y es el que se suma para revisar la capacidad del vehículo (nunca se usa el volumétrico).	Envio

5	String	tipoDestino	private	Guarda si el destino es urbano o departamental. Cada categoría lo usa para saber qué tarifa cobrar y cuántos días tarda la entrega.	Envio
6	String	estado	private	Guarda en qué estado está el envío: en bodega, asignado, en ruta, entregado o devuelto al remitente. Solo cambia por medio de los métodos de la clase, para que nunca quede en un estado inválido.	Envio
7	int	intentos	private	Cuenta cuántas veces ha salido el envío a ruta. Se compara con el máximo de intentos de su categoría para saber si se puede volver a asignar.	Envio
8	double	montoRegistro	private	Guarda lo que se le cobró al cliente cuando se registró el envío. Empieza en 0, por eso las cotizaciones y los envíos iniciales no suman ingresos.	Envio
9	double	cargosReintento	private	Va sumando los Q15 que se cobran por cada intento después del primero. Se suma cuando el despacho sale, no cuando se asigna el envío.	Envio
10	double	CARGO_REINTENTO	private static final	Guarda el 15, que es el cargo por cada intento adicional, para no tener que escribir ese número en varias partes del código.	Envio
11	double	TARIFA_BASE_URBANA	protected static final	Guarda la tarifa base de Q25 para destino urbano. Está en Envio porque la usan Paquete y CargaRefrigerada, y así no se repite en las dos clases.	Envio

12	double	TARIFA_BASE_DEPARTAMENTAL	protected static final	Guarda la tarifa base de Q40 para destino departamental. Igual que la urbana, está en la clase padre para no duplicarla.	Envio
13	double	PESO_MAXIMO	private static final	Guarda el 0.5, que es el peso máximo que puede tener un sobre. Se revisa en el constructor.	Sobre
14	double	COSTO_URBANO	private static final	Guarda los Q20 que cuesta un sobre con destino urbano, sin importar el peso.	Sobre
15	double	COSTO_DEPARTAMENTAL	private static final	Guarda los Q35 que cuesta un sobre con destino departamental.	Sobre
16	double	largo	private	Guarda el largo del paquete en centímetros. Tiene que ser mayor que 0 y se usa para calcular el peso volumétrico.	Paquete
17	double	ancho	private	Guarda el ancho del paquete en centímetros. Tiene que ser mayor que 0 y se usa para el peso volumétrico.	Paquete
18	double	alto	private	Guarda el alto del paquete en centímetros. Tiene que ser mayor que 0 y se usa para el peso volumétrico.	Paquete
19	double	PRECIO_POR_KG	private static final	Guarda los Q8 que se cobran por cada kilogramo facturable o fracción.	Paquete
20	double	FACTOR_VOLUMETRICO	private static final	Guarda el 5000 entre el que se divide largo × ancho × alto para sacar el peso volumétrico.	Paquete
21	double	valorDeclarado	private	Guarda el valor que el cliente declara del paquete frágil. Tiene que ser mayor que 0 y sobre ese valor se calcula el seguro.	PaqueteFragil

22	double	CARGO_EMBALAJE	private static final	Guarda los Q15 del embalaje especial que se le suman al costo de un paquete normal.	PaqueteFragil
23	double	PORCENTAJE_SEGURO	private static final	Guarda el 0.03 (3 %) que se aplica al valor declarado para calcular el seguro.	PaqueteFragil
24	double	temperaturaRequerida	private	Guarda la temperatura en grados Celsius a la que se tiene que transportar la carga. Tiene que estar entre −25 °C y 8 °C.	CargaRefrigerada
25	double	TEMPERATURA_MINIMA	private static final	Guarda el −25, que es la temperatura más baja permitida.	CargaRefrigerada
26	double	TEMPERATURA_MAXIMA	private static final	Guarda el 8, que es la temperatura más alta permitida.	CargaRefrigerada
27	double	PRECIO_POR_KG	private static final	Guarda los Q12 que se cobran por cada kilogramo de peso real de la carga.	CargaRefrigerada
28	double	CARGO_CADENA_FRIO	private static final	Guarda el cargo fijo de Q60 por mantener la cadena de frío.	CargaRefrigerada
29	String	placa	private	Guarda la placa que identifica al vehículo. No puede estar vacía ni repetirse.	Vehiculo
30	double	capacidadMaxima	private	Guarda cuántos kilogramos puede cargar el vehículo como máximo. Tiene que ser mayor que 0 y se compara con la suma de pesos reales del despacho.	Vehiculo
31	boolean	refrigerado	private	Guarda si el vehículo tiene refrigeración. Se usa para saber si puede llevar cargas refrigeradas.	Vehiculo
32	Despacho	despachoActual	private	Guarda el despacho sin cerrar que tiene el vehículo, o null si está libre. Sirve para que un vehículo no tenga más de un despacho abierto.	Vehiculo

33	int	numero	private	Guarda el número que identifica a cada despacho, para buscarlo cuando el usuario quiere asignar, retirar, despachar o registrar resultados.	Despacho
34	int	contadorDespachos	private static	Lleva la cuenta de cuántos despachos se han creado, para darle a cada uno un número distinto sin que el usuario lo escriba.	Despacho
35	Vehiculo	vehiculo	private	Guarda el vehículo que va a hacer el despacho. Se elige al crear el despacho y ya no cambia.	Despacho
36	ArrayList<Envio>	envios	private	Es la lista de envíos asignados al despacho. Crece o se hace más corta según se asignan o retiran envíos.	Despacho
37	String	estado	private	Guarda si el despacho está en preparación, en ruta o cerrado. Dependiendo del estado se permite o no asignar, retirar, salir o registrar resultados.	Despacho
38	ArrayList<Envio>	envios	private	Es la lista de todos los envíos registrados. Se guarda como Envio para tener envíos de cualquier categoría en la misma lista.	EmpresaMensajeria
39	ArrayList<Vehiculo>	vehiculos	private	Es la lista de los vehículos de la flotilla.	EmpresaMensajeria
40	ArrayList<Despacho>	despachos	private	Es la lista de todos los despachos creados, abiertos y cerrados. Se usa para buscarlos y para el reporte de despachos no cerrados.	EmpresaMensajeria
41	EmpresaMensajeria	empresa	private	Guarda la empresa sobre la que se trabaja, para que la vista le pida	Vista

				todas las operaciones que elija el usuario en el menú.	
42	Scanner	scanner	private	Guarda el lector que recibe lo que el usuario escribe en la consola, tanto textos como números.	Vista

2. Propiedades de métodos

Métodos					
#	retorno	nombre	visibilidad	motivo	Clase
1	—	Envio(String codigo, String remitente, String destinatario, double pesoReal, String tipoDestino)	protected	Es el constructor de la clase abstracta, así que no se usa con new directamente: lo llaman las subclases con super(...). Revisa que el código no esté vacío, que el peso sea mayor que 0 y que el destino sea urbano o departamental. Deja el envío en bodega, con 0 intentos y sin montos cobrados.	Envio
2	String	getCodigo()	public	Devuelve el código del envío. Se usa para buscarlo y para revisar que no se repita.	Envio
3	String	getRemitente()	public	Devuelve el nombre del remitente, para mostrarlo en las consultas.	Envio
4	String	getDestinatario()	public	Devuelve el nombre del destinatario, para mostrarlo en las consultas.	Envio
5	double	getPesoReal()	public	Devuelve el peso real. Despacho lo usa para sumar el peso cargado y revisar la capacidad del vehículo.	Envio
6	String	getTipoDestino()	public	Devuelve si el destino es urbano o departamental, para mostrarlo.	Envio
7	String	getEstado()	public	Devuelve el estado actual del envío. Se usa en el reporte de envíos por estado.	Envio
8	int	getIntentos()	public	Devuelve cuántos intentos de entrega lleva el envío, para mostrarlo al consultarlo.	Envio
9	double	getMontoRegistro()	public	Devuelve lo que se cobró al registrar el envío. Se usa para sumar los ingresos por registro.	Envio

10	double	getCargosReintento()	public	Devuelve lo que se ha cobrado por reintentos. Se usa para sumar los ingresos por reintentos.	Envio
11	boolean	esUrbano()	protected	Devuelve true si el destino es urbano. Lo usan las subclases para elegir su tarifa y sus días sin repetir la comparación de texto.	Envio
12	double	getTarifaBase()	protected	Devuelve Q25 si el destino es urbano o Q40 si es departamental. Lo usan Paquete y CargaRefrigerada para no repetir la regla.	Envio
13	double	calcularCosto()	public abstract	Devuelve el costo del envío. Es abstracto porque cada categoría cobra distinto y Envio no sabe cómo calcularlo; cada subclase lo sobrescribe.	Envio
14	int	calcularDiasEntrega()	public abstract	Devuelve los días hábiles estimados de entrega. Es abstracto porque cada categoría tiene sus propios días.	Envio
15	int	getMaxIntentos()	public abstract	Devuelve cuántos intentos de entrega permite la categoría. Es abstracto porque es la política de reintentos de cada subclase.	Envio
16	String	getCategoria()	public abstract	Devuelve el nombre de la categoría. Se usa en los reportes para agrupar los envíos sin tener que preguntar el tipo del objeto con instanceof.	Envio
17	boolean	requiereRefrigeracion()	public	Devuelve false por defecto, porque la mayoría de envíos puede ir en cualquier vehículo. Solo CargaRefrigerada lo sobrescribe.	Envio
18	boolean	puedeAsignarse()	public	Revisa que el envío esté en bodega y que todavía no haya agotado sus intentos. Despacho lo usa antes de asignarlo.	Envio

19	void	cobrarRegistro()	public	Guarda en montoRegistro el resultado de calcularCosto(). Solo se llama cuando el cliente acepta y el envío se registra, por eso una cotización no cobra nada.	Envio
20	void	asignar()	public	Cambia el estado a asignado. Si el envío no puede asignarse lanza un error y no cambia nada.	Envio
21	void	retirar()	public	Regresa el envío a bodega cuando se retira de un despacho en preparación. No cambia los intentos ni los montos.	Envio
22	double	salirARuta()	public	Suma un intento y pasa el envío a en ruta. Si no es el primer intento agrega Q15 a cargosReintento. Devuelve el cargo que se generó (0 o 15).	Envio
23	void	registrarEntregaExitosa()	public	Cambia el estado de en ruta a entregado. Si el envío no estaba en ruta lanza un error.	Envio
24	void	registrarEntregaFallida()	public	Si el envío ya llegó al máximo de intentos de su categoría queda devuelto al remitente; si no, regresa a bodega para asignarlo a otro despacho.	Envio
25	boolean	estaEnRuta()	public	Devuelve true si el envío está en ruta, o sea que todavía no tiene resultado. Despacho lo usa para saber si ya puede cerrarse.	Envio
26	String	toString()	public	Junta el código, categoría, remitente, destinatario, peso, destino, estado e intentos en un solo texto para mostrar el envío completo.	Envio
27	—	Sobre(String codigo, String remitente, String destinatario, double pesoReal, String tipoDestino)	public	Crea un sobre. Primero llama a super(...) para que Envio guarde y valide los datos comunes, y luego revisa que el peso no pase de 0.5 kg.	Sobre

28	double	calcularCosto()	public @Override	Devuelve Q20 si el destino es urbano o Q35 si es departamental, sin importar el peso.	Sobre
29	int	calcularDiasEntrega()	public @Override	Devuelve 1 día si es urbano y 2 si es departamental.	Sobre
30	int	getMaxIntentos()	public @Override	Devuelve 3, que son los intentos permitidos para un sobre.	Sobre
31	String	getCategoria()	public @Override	Devuelve "Sobre" para los reportes.	Sobre
32	—	Paquete(String codigo, String remitente, String destinatario, double pesoReal, String tipoDestino, double largo, double ancho, double alto)	public	Crea un paquete. Llama a super(...) con los datos comunes y después revisa que el largo, ancho y alto sean mayores que 0 antes de guardarlos.	Paquete
33	double	getLargo()	public	Devuelve el largo del paquete, para mostrarlo.	Paquete
34	double	getAncho()	public	Devuelve el ancho del paquete, para mostrarlo.	Paquete
35	double	getAlto()	public	Devuelve el alto del paquete, para mostrarlo.	Paquete
36	double	calcularPesoVolumetrico()	public	Calcula largo × ancho × alto / 5000. Por ejemplo, 40×30×20 da 4.8 kg.	Paquete
37	int	calcularPesoFacturable()	public	Toma el mayor entre el peso real y el volumétrico y lo redondea hacia arriba, porque se cobra por kilogramo o fracción (4.8 kg se cobra como 5).	Paquete
38	double	calcularCosto()	public @Override	Devuelve la tarifa base según el destino más Q8 por cada kilogramo facturable. Con el ejemplo del enunciado da Q25 + Q40 = Q65.	Paquete
39	int	calcularDiasEntrega()	public @Override	Devuelve 2 días si es urbano y 4 si es departamental.	Paquete
40	int	getMaxIntentos()	public @Override	Devuelve 3, que son los intentos permitidos para un paquete.	Paquete
41	String	getCategoria()	public @Override	Devuelve "Paquete" para los reportes.	Paquete

42	String	toString()	public @Override	Usa super.toString() y le agrega las medidas y el peso volumétrico, para no repetir los datos comunes.	Paquete
43	—	PaqueteFragil(String codigo, String remitente, String destinatario, double pesoReal, String tipoDestino, double largo, double ancho, double alto, double valorDeclarado)	public	Crea un paquete frágil. Llama a super(...) de Paquete, que a su vez llama al de Envio, así cada constructor valida sus propios datos. Después revisa que el valor declarado sea mayor que 0.	PaqueteFragil
44	double	getValorDeclarado()	public	Devuelve el valor declarado, para mostrarlo.	PaqueteFragil
45	double	calcularSeguro()	public	Calcula el 3 % del valor declarado. Con Q1,200 da Q36.	PaqueteFragil
46	double	calcularCosto()	public @Override	Usa super.calcularCosto() para sacar el costo de un paquete normal y le suma Q15 de embalaje y el seguro. Con el ejemplo da Q65 + Q15 + Q36 = Q116.	PaqueteFragil
47	int	calcularDiasEntrega()	public @Override	Devuelve super.calcularDiasEntrega() + 1, o sea un día más que un paquete normal con el mismo destino.	PaqueteFragil
48	int	getMaxIntentos()	public @Override	Devuelve 2, que son los intentos permitidos para un paquete frágil.	PaqueteFragil
49	String	getCategoria()	public @Override	Devuelve "Paquete frágil" para los reportes.	PaqueteFragil
50	String	toString()	public @Override	Usa super.toString() y le agrega el valor declarado y el seguro.	PaqueteFragil
51	—	CargaRefrigerada(String codigo, String remitente, String destinatario, double pesoReal, String tipoDestino, double temperaturaRequerida)	public	Crea una carga refrigerada. Llama a super(...) y luego revisa que la temperatura esté entre −25 °C y 8 °C y que el destino sea urbano, porque no se aceptan departamentales.	CargaRefrigerada
52	double	getTemperaturaRequerida()	public	Devuelve la temperatura requerida, para mostrarla.	CargaRefrigerada

53	double	calcularCosto()	public @Override	Devuelve la tarifa base según el destino más Q12 por kilogramo de peso real y Q60 por la cadena de frío.	CargaRefrigerada
54	int	calcularDiasEntrega()	public @Override	Devuelve siempre 1 día.	CargaRefrigerada
55	int	getMaxIntentos()	public @Override	Devuelve 1, porque después del primer intento la cadena de frío ya se rompió.	CargaRefrigerada
56	boolean	requiereRefrigeracion()	public @Override	Devuelve true, para que el vehículo sepa que esta carga solo puede ir en un vehículo refrigerado.	CargaRefrigerada
57	String	getCategoria()	public @Override	Devuelve "Carga refrigerada" para los reportes.	CargaRefrigerada
58	String	toString()	public @Override	Usa super.toString() y le agrega la temperatura requerida.	CargaRefrigerada
59	—	Vehiculo(String placa, double capacidadMaxima, boolean refrigerado)	public	Crea un vehículo con su placa, capacidad y si es refrigerado. Revisa que la placa no esté vacía y que la capacidad sea mayor que 0. Empieza libre, sin despacho.	Vehiculo
60	String	getPlaca()	public	Devuelve la placa del vehículo. Se usa para buscarlo y para revisar que no se repita.	Vehiculo
61	double	getCapacidadMaxima()	public	Devuelve la capacidad máxima. Despacho la usa para revisar que no se pase del peso.	Vehiculo
62	boolean	isRefrigerado()	public	Devuelve si el vehículo tiene refrigeración, para mostrarlo.	Vehiculo
63	boolean	estaLibre()	public	Devuelve true si el vehículo no tiene ningún despacho sin cerrar.	Vehiculo
64	boolean	puedeTransportar(Envio envio)	public	Revisa si el envío es compatible con el vehículo: si envio.requiereRefrigeracion() es true, el vehículo tiene que ser refrigerado.	Vehiculo

				Aquí se valida la compatibilidad sin preguntar la categoría.	
65	void	ocupar(Despacho despacho)	public	Guarda el despacho que va a hacer el vehículo. Si ya tiene uno sin cerrar lanza un error.	Vehiculo
66	void	liberar()	public	Deja despachoActual en null cuando el despacho se cierra, para que el vehículo pueda usarse otra vez.	Vehiculo
67	String	toString()	public	Junta la placa, capacidad, si es refrigerado y si está libre en un solo texto.	Vehiculo
68	—	Despacho(Vehiculo vehiculo)	public	Crea un despacho para el vehículo indicado. Revisa que el vehículo esté libre, le da el siguiente número con el contador, crea la lista vacía, lo deja en preparación y llama a vehiculo.ocupar(this).	Despacho
69	int	getNumero()	public	Devuelve el número del despacho, para buscarlo.	Despacho
70	Vehiculo	getVehiculo()	public	Devuelve el vehículo del despacho, para mostrarlo en el reporte.	Despacho
71	String	getEstado()	public	Devuelve el estado del despacho.	Despacho
72	double	calcularPesoCargado()	public	Suma los pesos reales de todos los envíos asignados. No usa el peso volumétrico.	Despacho
73	double	calcularCapacidadDisponible()	public	Devuelve la capacidad del vehículo menos el peso cargado. Se usa en el reporte de despachos abiertos.	Despacho
74	void	asignarEnvio(Envio envio)	public	Asigna un envío al despacho. Antes revisa que el despacho esté en preparación, que el envío pueda asignarse, que no se pase la capacidad y que el vehículo pueda transportarlo. Si algo falla lanza un error sin cambiar nada.	Despacho

75	void	retirarEnvio(String codigo)	public	Quita un envío del despacho mientras está en preparación y lo regresa a bodega. No afecta los ingresos.	Despacho
76	double	salir()	public	Hace salir el despacho. Revisa que no esté vacío y que esté en preparación, llama a salirARuta() de cada envío y pasa a en ruta. Aquí se registra el cargo por reintento. Devuelve el total de cargos generados.	Despacho
77	void	registrarResultado(String codigo, boolean entregado)	public	Registra si un envío del despacho fue entregado o falló. Cuando todos tienen resultado, cierra el despacho.	Despacho
78	Envio	buscarEnvio(String codigo)	private	Recorre la lista del despacho y devuelve el envío con ese código, o null si no está.	Despacho
79	boolean	todosConResultado()	private	Revisa si ya ningún envío está en ruta, o sea que todos tienen resultado registrado.	Despacho
80	void	cerrar()	private	Cambia el estado a cerrado y llama a vehiculo.liberar() para que el vehículo quede libre.	Despacho
81	boolean	estaCerrado()	public	Devuelve true si el despacho está cerrado. Se usa para el reporte de despachos no cerrados.	Despacho
82	String	toString()	public	Junta el número, vehículo, estado, peso cargado, capacidad disponible y los envíos en un solo texto.	Despacho
83	—	EmpresaMensajeria()	public	Crea la empresa con las listas de envíos, vehículos y despachos vacías.	EmpresaMensajeria
84	String	cotizar(Envio envio)	public	Devuelve el costo con dos decimales y los días de entrega del envío. No lo agrega a la lista ni cobra nada, así la cotización no cambia los ingresos.	EmpresaMensajeria
85	void	registrarEnvio(Envio envio)	public	Revisa que el código no exista, agrega el envío a la lista y llama a	EmpresaMensajeria

				cobrarRegistro() para cobrar el monto completo. El envío queda en bodega.	
86	void	agregarEnvioInicial(Envio envio)	public	Agrega un envío de los datos iniciales sin cobrarlo, porque el programa tiene que empezar sin ingresos registrados.	EmpresaMensajeria
87	boolean	existeCodigoEnvio(String codigo)	private	Revisa si algún envío ya tiene ese código, para no registrar dos iguales.	EmpresaMensajeria
88	Envio	buscarEnvio(String codigo)	public	Recorre la lista y devuelve el envío con ese código, o null si no existe.	EmpresaMensajeria
89	void	registrarVehiculo(Vehiculo vehiculo)	public	Agrega un vehículo a la flotilla. Antes revisa que la placa no esté repetida.	EmpresaMensajeria
90	boolean	existePlaca(String placa)	private	Revisa si algún vehículo ya tiene esa placa.	EmpresaMensajeria
91	Vehiculo	buscarVehiculo(String placa)	public	Devuelve el vehículo con esa placa, o null si no existe.	EmpresaMensajeria
92	Despacho	crearDespacho(String placa)	public	Busca el vehículo, crea un despacho nuevo con él y lo agrega a la lista. Si el vehículo no existe o no está libre lanza un error.	EmpresaMensajeria
93	Despacho	buscarDespacho(int numero)	public	Devuelve el despacho con ese número, o null si no existe.	EmpresaMensajeria
94	void	asignarEnvio(int numeroDespacho, String codigoEnvio)	public	Busca el despacho y el envío y le pide al despacho que lo asigne. Las validaciones las hace Despacho.	EmpresaMensajeria
95	void	retirarEnvio(int numeroDespacho, String codigoEnvio)	public	Busca el despacho y le pide que retire el envío.	EmpresaMensajeria
96	void	hacerSalirDespacho(int numeroDespacho)	public	Busca el despacho y llama a salir(). Los cargos por reintento quedan guardados en cada envío.	EmpresaMensajeria
97	void	registrarResultado(int numeroDespacho, String codigoEnvio, boolean entregado)	public	Busca el despacho y le pide que registre el resultado de ese envío.	EmpresaMensajeria
98	String	listarEnvios()	public	Arma un texto con todos los envíos registrados. Si no hay ninguno lo avisa.	EmpresaMensajeria

99	String	listarVehiculos()	public	Arma un texto con todos los vehículos de la flotilla.	EmpresaMensajeria
100	String	reporteEnviosPorCategoriaYEstado()	public	Cuenta cuántos envíos hay de cada categoría y en cada estado usando getCategoria() y getEstado(), sin instanceof.	EmpresaMensajeria
101	double	calcularIngresosRegistro()	public	Suma el montoRegistro de todos los envíos.	EmpresaMensajeria
102	double	calcularIngresosReintentos()	public	Suma los cargosReintento de todos los envíos.	EmpresaMensajeria
103	double	calcularIngresoTotal()	public	Suma los ingresos por registro y por reintentos.	EmpresaMensajeria
104	String	reporteIngresosPorCategoria()	public	Arma un texto con el ingreso de cada categoría, agrupando con getCategoria().	EmpresaMensajeria
105	String	reporteDespachosAbiertos()	public	Arma un texto con los despachos que no están cerrados, con su vehículo, peso cargado y capacidad disponible.	EmpresaMensajeria
106	—	Vista()	public	Prepara la vista creando el lector de la consola y la empresa, y carga los datos iniciales.	Vista
107	void	mostrarMenu()	public	Muestra las opciones del menú y lee la que elija el usuario. Se repite hasta que el usuario escoge salir. Atrapa los errores de cada operación para mostrar un mensaje claro sin cerrar el programa.	Vista
108	void	cargarDatosIniciales()	private	Crea al menos dos envíos de cada categoría en bodega (incluyendo un paquete con peso volumétrico mayor que el real) y tres vehículos, al menos uno refrigerado y uno no.	Vista
109	Envio	leerDatosEnvio()	private	Pide la categoría y los datos del envío y crea el objeto de la subclase que corresponde. Es el único lugar donde	Vista

				se elige la clase, y se usa tanto para cotizar como para registrar.	
110	void	cotizarEnvio()	private	Usa leerDatosEnvio() y muestra el costo y los días que devuelve la empresa, sin registrar nada.	Vista
111	void	registrarEnvio()	private	Usa leerDatosEnvio(), muestra la cotización y, si el cliente acepta, le pide a la empresa que registre el envío.	Vista
112	void	registrarVehiculo()	private	Pide la placa, la capacidad y si es refrigerado, y le pide a la empresa que registre el vehículo.	Vista
113	void	crearDespacho()	private	Pide la placa del vehículo y le pide a la empresa que cree el despacho. Muestra el número asignado.	Vista
114	void	asignarEnvio()	private	Pide el número de despacho y el código del envío, y le pide a la empresa que lo asigne.	Vista
115	void	retirarEnvio()	private	Pide el número de despacho y el código del envío que se quiere retirar.	Vista
116	void	salirDespacho()	private	Pide el número de despacho y le pide a la empresa que lo haga salir.	Vista
117	void	registrarResultados()	private	Pide el número de despacho y, para cada envío en ruta, pregunta si fue entregado o falló.	Vista
118	void	consultarEnvios()	private	Muestra la lista de envíos que devuelve la empresa.	Vista
119	void	consultarVehiculos()	private	Muestra la lista de vehículos que devuelve la empresa.	Vista
120	void	mostrarReportes()	private	Muestra los reportes de envíos por categoría y estado, de ingresos (total, por registro, por reintentos y por categoría) y de despachos abiertos, con montos a dos decimales.	Vista

121	int	leerEntero(String mensaje)	private	Muestra el mensaje y lee un número entero. Si el usuario escribe algo que no es número, avisa y vuelve a pedirlo sin cerrar el programa.	Vista
122	double	leerDecimal(String mensaje)	private	Igual que leerEntero, pero para números con decimales como pesos, medidas y temperaturas.	Vista
123	String	leerTexto(String mensaje)	private	Muestra el mensaje y lee una línea de texto, como códigos, nombres o placas.	Vista
124	void	main(String[] args)	public static	Es el punto de entrada del programa. Crea la Vista y llama a mostrarMenu().	Main

3. Requisitos funcionales

- **Información:** los envíos (datos comunes y los propios de cada categoría, estado, intentos y montos cobrados), los vehículos de la flotilla y los despachos. Los ingresos se calculan a partir de los montos de cada envío.
- **Operaciones:** cotizar y registrar envíos, registrar vehículos, crear despachos, asignar y retirar envíos, hacer salir un despacho, registrar resultados de entrega, consultar envíos y vehículos, y generar reportes de envíos, ingresos y despachos abiertos.
- **Condiciones:** cada operación valida los datos y el estado de los objetos involucrados (datos válidos por categoría, códigos y placas únicos, despacho en preparación, capacidad y compatibilidad del vehículo, intentos disponibles). Si algo falla, se rechaza con un mensaje claro sin modificar el estado ni los ingresos.

4. Decisiones que dependen de la categoría del envío

Cada decisión que cambia según la categoría se declara en Envio y la resuelve la subclase que tiene los datos. Así, Despacho, Vehiculo y EmpresaMensajeria trabajan siempre con Envio, sin usar instanceof.

- **Validar los datos propios:** el constructor de cada subclase.
- **Costo y días de entrega:** calcularCosto() y calcularDiasEntrega() en cada subclase; PaqueteFragil reutiliza los de Paquete.
- **Peso volumétrico y facturable:** Paquete, la única clase con medidas.
- **Máximo de intentos:** getMaxIntentos() en cada subclase; Envio lo usa para decidir si el envío vuelve a bodega o se devuelve al remitente.
- **Vehículo refrigerado:** requiereRefrigeracion(), que en Envio devuelve false y en CargaRefrigerada true; Vehiculo lo consulta para validar la compatibilidad.
- **Nombre en los reportes:** getCategoria() en cada subclase.
- **Qué subclase crear:** Vista, el único punto que conoce la clase concreta.

5. Estados de un envío y de un despacho

Estado inicial	Operación	Estado final
—	Registrar envío (o cargarlo en los datos iniciales)	En bodega
En bodega	Asignar a un despacho	Asignado
Asignado	Retirar del despacho	En bodega
Asignado	Salida del despacho	En ruta
En ruta	Registrar resultado: entregado	Entregado
En ruta	Entrega fallida con intentos disponibles	En bodega
En ruta	Entrega fallida sin intentos disponibles	Devuelto al remitente

Un despacho tiene tres estados: en preparación, en ruta y cerrado. Cerrado es el estado final. Cualquier operación que no aparezca en estas tablas para el estado actual se rechaza.

Estado inicial	Operación	Estado final
—	Crear despacho	En preparación
En preparación	Asignar o retirar envíos	En preparación
En preparación	Hacer salir el despacho	En ruta
En ruta	Registrar el resultado de un envío	En ruta
En ruta	Registrar el resultado del último envío	Cerrado

6. Clases, responsabilidades y valores iniciales

Envio (abstracta) guarda los datos comunes, controla el estado, los intentos y los cobros, y declara los métodos que cada categoría implementa. Sobre, Paquete y CargaRefrigerada heredan de Envio, y PaqueteFragil de Paquete; cada una aplica sus propias reglas. Vehiculo decide si puede transportar un envío y si está libre; Despacho administra la asignación, la salida, los resultados y el cierre; EmpresaMensajeria guarda las listas, evita duplicados y genera los reportes; Vista maneja la consola y Main inicia el programa.

EmpresaMensajeria tiene 0 envíos, vehículos y despachos. Cada Despacho tiene 1 Vehiculo y 0 envíos, y cada Vehiculo tiene como máximo un despacho abierto.

Cada objeto recibe sus datos por el constructor y queda en un estado inicial válido: el envío en bodega con 0 intentos, el vehículo libre y el despacho en preparación. En la jerarquía, los constructores colaboran en cadena: PaqueteFragil llama a super de Paquete, que llama al de Envio. Así, Envio valida los datos comunes, Paquete las medidas y PaqueteFragil el valor declarado, sin repetir validaciones. Si un dato es inválido se lanza una excepción y el objeto no se agrega al sistema.

7. Relación entre el paquete frágil y el paquete

PaqueteFragil hereda de Paquete porque un paquete frágil es un paquete con embalaje y seguro adicionales. Así reutiliza las medidas y el peso facturable, calcula su costo y sus días a partir de los de Paquete (super) y solo sobrescribe lo que cambia. La desventaja es que queda acoplado a Paquete y que instanceof Paquete también lo incluiría, por eso los reportes agrupan con getCategoria().

- **Heredar de Envio:** más independiente, pero duplica las medidas y las reglas de cálculo.
- **Composición:** menos acoplamiento, pero duplica los datos comunes y agrega métodos que solo delegan.
- **Atributo booleano en Paquete:** una clase menos, pero llena Paquete de condicionales y de datos que no siempre aplican.

8. Impacto de agregar una nueva categoría

Basta con crear una subclase de Envio que implemente sus reglas (costo, días, intentos, categoría y, si aplica, requiereRefrigeracion()) y agregar la opción en Vista. Despachos y reportes no cambian, porque trabajan con Envio y agrupan por getCategoria(). Solo si la categoría necesitara algo que hoy no existe, como otro tipo de vehículo, habría que agregar un método con valor por defecto en Envio y su revisión en Vehiculo.